

```

//w
/* This method will schedule a timeout at WaitReplyTimeout interval
 * in the future, unless a timer is already running for the cache,
 * in which case this method does nothing.
 */
void StartWaitReplyTimer (void);
//w
/* (brief) Do lookup in the ARP cache against an IP address
 * (param Destination The destination IPv4 address to lookup the MAC address
 * of
 * (return An ArpCache::Entry with info about layer 2
 */
ArpCache::Entry *Lookup (Ipv4Address destination);
//w
/* (brief) Add an ipv4address to this ARP cache
 */

```

Figure 1: Doxygen comments inside an ns-3 source file

If you feel that you are ready to install ns-3, then let us begin. First, create a new directory in your Linux file system called 'ns' (or any other name that you prefer). Then copy the file into the directory 'ns'. After copying, you have to extract (unzip) the file. To do so, either right click on the file with your mouse and select the menu item called 'extract here' – or something similar—or execute the following command on a terminal. Set the path of the terminal to the directory 'ns'.

```
tar xjf ns-allinone-3.22.tar.bz2
```

Now you have the unzipped file inside the directory called *ns-allinone-3.22* in your current directory 'ns'. Move to the directory *ns-allinone-3.22/ns-3.22* by typing the following command in a terminal:

```
cd ns-allinone-3.22/ns-3.22
```

To start the installation of ns-3, type the following commands on the terminal:

```
./waf configure
./waf clean
./waf --build-profile=debug --enable-examples --enable-tests
configure
```

This will install ns-3 with the build profile *debug* enabled. If you want to enable the build profile *optimized*, then use the following command:

```
./waf --build-profile=optimized --enable-examples --enable-
tests configure
```

But remember that many of the example scripts provided by ns-3 may not be available with the build profile *optimized*. If the build profile used is *debug*, you can execute the *Hello World* equivalent of ns-3 by typing the following command on the terminal:

```
./waf --run hello-simulator
```

The output of the script executed is *Hello World*. Thus, we have executed our first ns-3 script—a milestone in our long journey.

The basics of socket programming

Now that we have installed ns-3 and executed our first ns-3 script, the next logical step is to learn socket programming. To do so, let us begin by discussing the TCP/IP model of a network, which divides the different functionalities associated with a computer network into a number of layers. The different layers are the application layer, transport layer, Internet layer and the link layer (the link layer in the TCP/IP model also includes the physical layer of the OSI model — another network model).

The process of sending a text file from one system (node) to another can be summarised as follows. At the sender side, the message is first processed by the application layer protocol, FTP, which deals with the user aspects, followed by the transport layer protocol, TCP, responsible for process-to-process connectivity (selecting the correct process from a number of processes at the receiver node). This is followed by the network layer protocol, IP, which is responsible for source-to-destination connectivity (selecting the receiver node from a number of nodes in a network), and finally, a link layer protocol, say ARP, which helps in finding the next node by converting the IP address to a MAC address, is called. At the receiver side, all these layers operate on the message in the reverse order. Even though there are four layers associated with network communication, in socket programming, we mostly worry about the two layers in the middle—the transport layer and the network layer.

The protocol data units associated with these layers are as follows: frames in the link layer, packets in the network layer, segments (for TCP) or datagram (for UDP) in the transport layer, and messages in the application layer. Similarly, three of these layers have addresses associated with them — the port address (16 bit) associated with the transport layer, the IP address (IPv4 address is 32 bit and IPv6 address is 128 bit) associated with the network layer, and the MAC address (48 bit) associated with the link layer. So we need at least a few of these addresses for communication, depending on the nature and placement of the two nodes that are communicating.

Now that we are familiar with the layered architecture of a network and the different addresses associated with each layer, the next important question to be answered is, "What is a socket?" It is a mechanism used to realise client-server or peer-to-peer communication in a network. It is a protocol-independent method to create a connection between two processes residing in the same computer or different computers in a network. This clearly highlights one major difference between ns-2 and ns-3. In the latter, real data is sent between two processes to analyse the network's behaviour, whereas in ns-2, real data flow does not happen.

Socket programming is used in ns-3 to set up inter-process communication between two processes such that actual data can be sent between them. This data flow is monitored by various ns-3 modules and summarised into a log file called the trace file. By analysing the trace file, you obtain your results.

Now let us look at how to set up a connection between two